Universidade do Minho

Ano Letivo: 2026/27

Sistemas Conexionistas para Aprendizagem Automática

PL3 – PyTorch



Licenciatura em Ciência de Dados, 1º Semestre, 3º Ano

Sumário

1 PyTorch

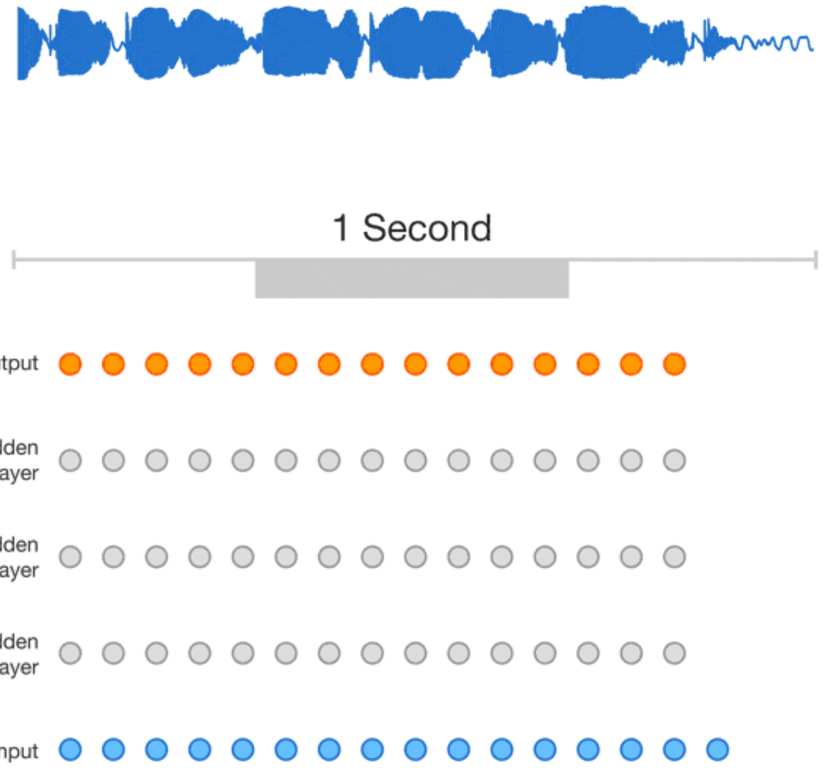
2 Tensores

3 Autograd

4 Aleatoriedade

Projetos de Deep Learning

Texto para Discurso



(https://deepmind.com/blog/wavenet-generative-model-raw-audio/)

Projetos de Deep Learning

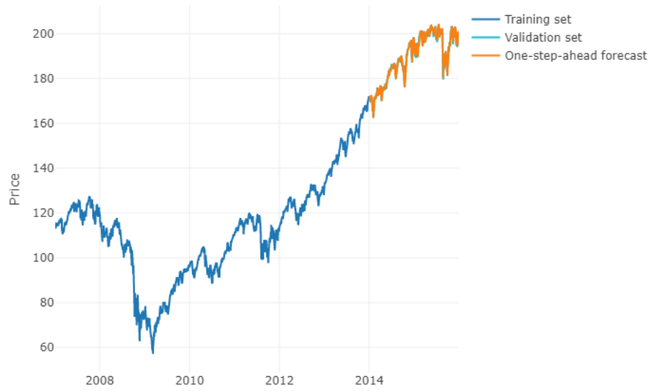
Criação de rostos



(https://www.thispersondoesnotexist.com/)

Projetos de Deep Learning

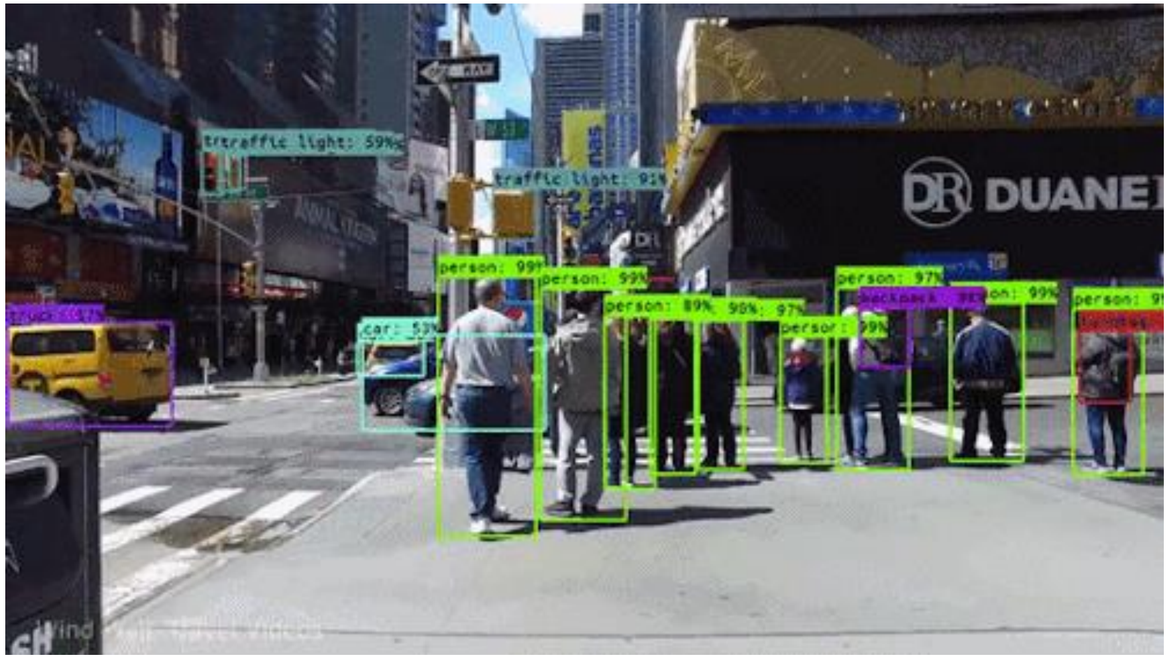
Outros projetos





ANNs também estão a ser usadas para:

- Modelos de linguagem (LLMs)
- Geração de imagens (difusão)
- Text-to-speech
- Biologia (AlphaFold)
- Image Classification
- Object Detection
- Time Series
- …



PyTorch

Instalação PyTorch

Criar um ambiente novo com o conda:

- *conda create -n scaa python=3.12*
  conda activate scaa

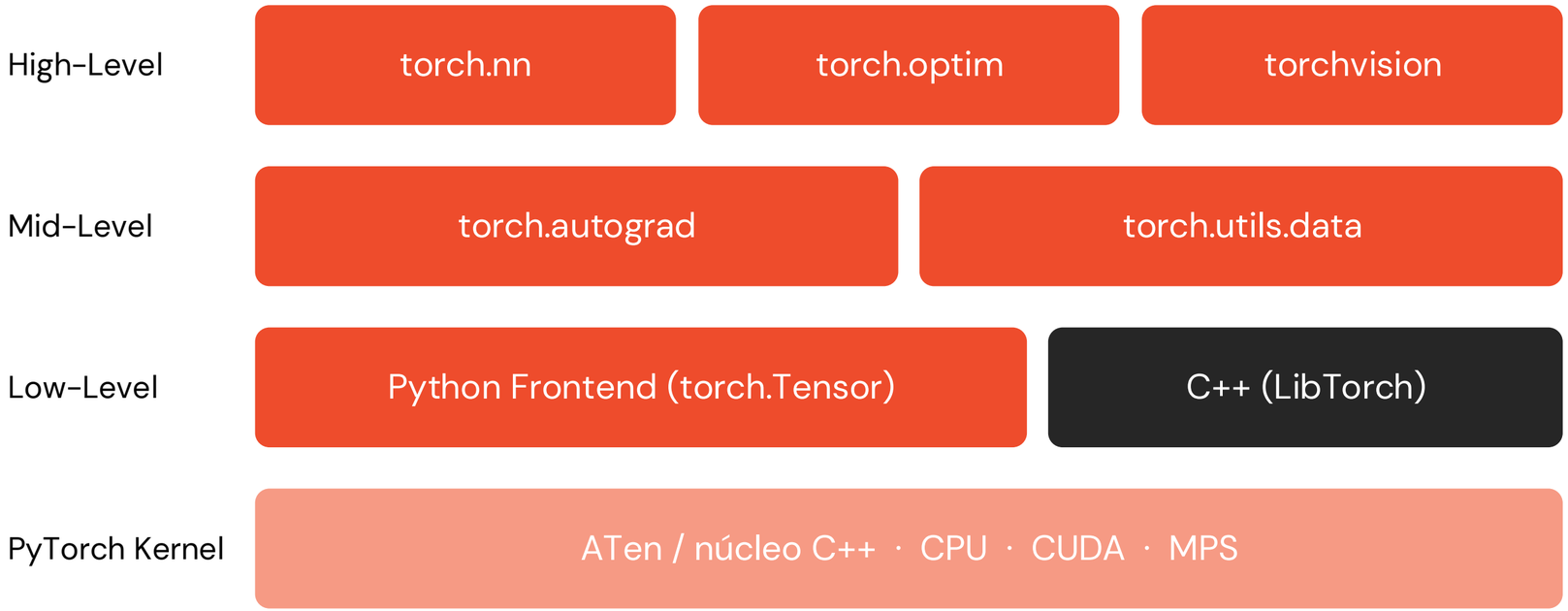
Instalar com o pip (dentro do ambiente):

- *pip install torch==2.14.1 torchvision==0.29.1*

Em relação ao **Python**, vamos instalar a versão 3.12!

NÃO TRABALHAR NO AMBIENTE DE RAÍZ!!! Criar um ambiente novo!

PyTorch Stack de Programação



Arquitetura do PyTorch

O PyTorch **define** a computação **à medida que a executa** (*define-by-run*):

- **Tensores:** Os tensores são as unidades fundamentais de dados no PyTorch. São matrizes ou conjuntos multidimensionais utilizados para armazenar dados.

- **Grafo:** Um grafo representa uma computação como um fluxo de tensores através de uma série de operações. No PyTorch, o grafo é construído dinamicamente, à medida que cada operação (multiplicação, adição, ativação, ...) é executada sobre os tensores de entrada.

- **Autograd:** O autograd percorre o grafo no sentido inverso e calcula os gradientes automaticamente. Não existem sessões.

Eager Execution

- O PyTorch apresenta-nos um ambiente de programação que **avalia as operações de imediato**, **sem** a **necessidade** de construir **grafos** manualmente.
- Com a **execução imediata** (***eager execution***), as operações do PyTorch comportam-se como código Python normal, ou seja, avaliam e devolvem os seus valores **imediatamente para o Python**.
- As **operações devolvem valores concretos**, que podem ser inspecionados com print() ou com um debugger.
- No **PyTorch**, a **Execução Eager** é o modo por omissão: não é preciso ativá-la.

Como verificar a instalação do PyTorch (e se há GPU)?

```
import torch

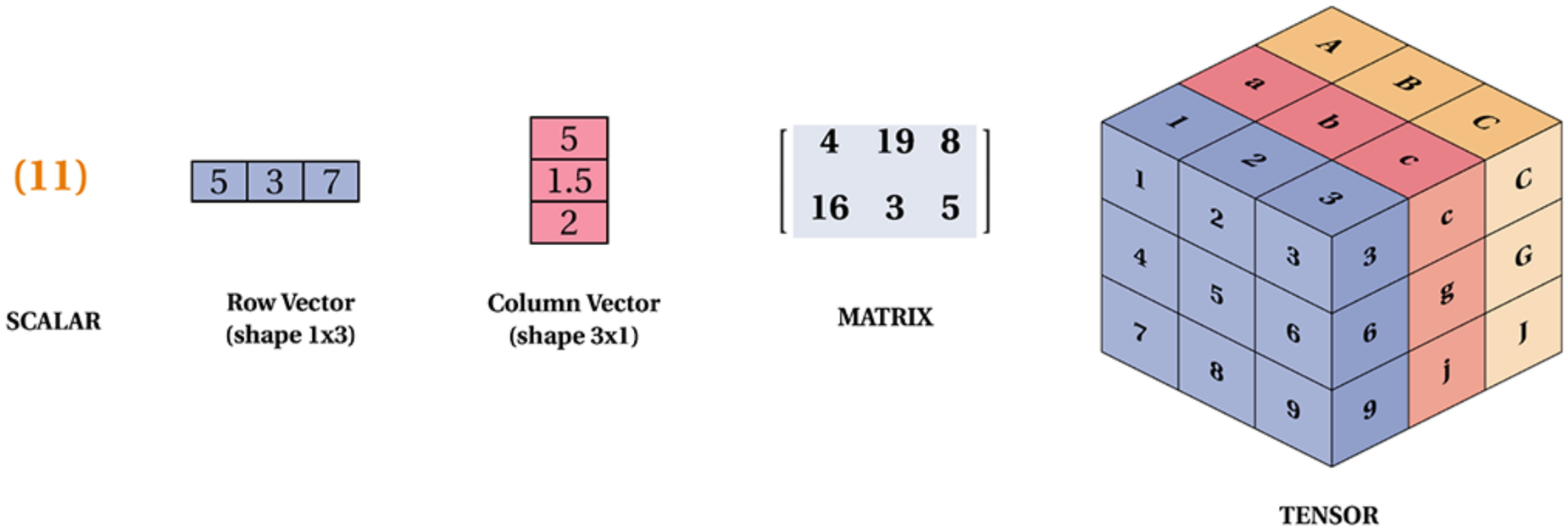
print('PyTorch:', torch.__version__)
print(torch.cuda.is_available())
```



```
PyTorch: 2.14.1
False
```

Tensores

- São a **estrutura de dados fundamental** do PyTorch.
- Um tensor é uma **matriz n-dimensional** que pode ter **nenhuma** dimensão (escalar), **uma** dimensão (vetor), **duas** dimensões (matriz) ou **mais** dimensões.



- Os tensores podem ter operações aplicadas sobre eles, como a adição, a subtração, o produto escalar, o produto vetorial, etc.

Tensores

- Cada torch.Tensor possui:
  - **Dimensão (rank/ndim)** – informa o número de dimensões do tensor
  - **Tipo de dado (dtype)** - informa o tipo de todos os elementos no tensor
  - **Forma (shape)** - informa a estrutura do tensor, i.e., o número de elementos ao longo de cada um de seus eixos
  - **Dispositivo (device)** - informa onde o tensor está armazenado (CPU ou GPU)

```
import torch
t = torch.tensor([[1, 2], [3, 4]])
```

Tensores

Rank/Degree

- O **Rank** ou **Degree** é o **número** de **eixos** tensores. Um **escalar** tem rank **0**, um **vetor** tem rank **1**, ...

Rank	Descrição	Implementação	Exemplo
0	Escalar (apenas magnitude)	`rank_0_tensor = torch.tensor(4)`	4
1	Vetor (magnitude e direção)	`rank_1_tensor = torch.tensor([2, 3, 4])`	[2,3,4]
2	Matriz (tabela de números)	`rank_2_tensor = torch.tensor(` `  [[1, 2], [5, 6]])`	[[1 2] [5 6]]

3	3-Tensor (cubo de números)	`rank_3_tensor = torch.tensor([`
		`  [[0, 1, 2, 3, 4],`
		`  [5, 6, 7, 8, 9]],`
		`  [[10, 11, 12, 13, 14],`
		`  [15, 16, 17, 18, 19]],`
		`  [[20, 21, 22, 23, 24],`
		`  [25, 26, 27, 28, 29]],])`
			[[[0 1 2 3 4]
			[5 6 7 8 9]]
			
			[[10 11 12 13 14]
			[15 16 17 18 19]]
			
			[[20 21 22 23 24]
			[25 26 27 28 29]]]

n	N-Tensor (por aí fora)	...	...

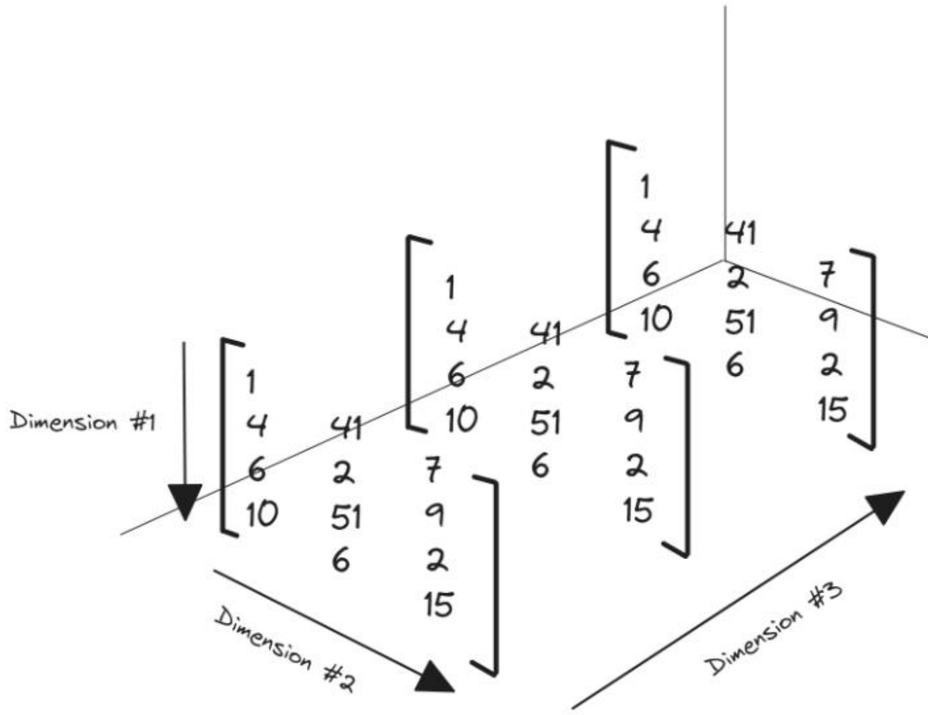
```
print(t.ndim)  # para obter o rank do tensor
```

PyTorch

Tensores

Rank/Degree

- O **Rank** ou **Degree** é o **número** de **eixos** tensores. Um **escalar** tem rank **0**, um **vetor** tem rank **1,** ...



Tensores

Shape

- A **Shape** é uma tupla de inteiros que se refere ao **layout** ou **estrutura** de um tensor.
- Descreve **quantos elementos** estão ao **longo** de cada **eixo** do tensor.

Rank	Descrição	Shape	Exemplo
0	Escalar (apenas magnitude)	[]	4
1	Vetor (magnitude e direção)	[3]	[2,3,4]
2	Matriz (tabela de números)	[2, 2]	[[1 2] [5 6]]
3	3-Tensor (cubo de números)	[3, 2, 5]	[[[0 1 2 3 4] [5 6 7 8 9]] [[10 11 12 13 14] [15 16 17 18 19]] [[20 21 22 23 24] [25 26 27 28 29]]]
n	N-Tensor (por aí fora)	...[D0, D1, D2, ... DN-1)	...

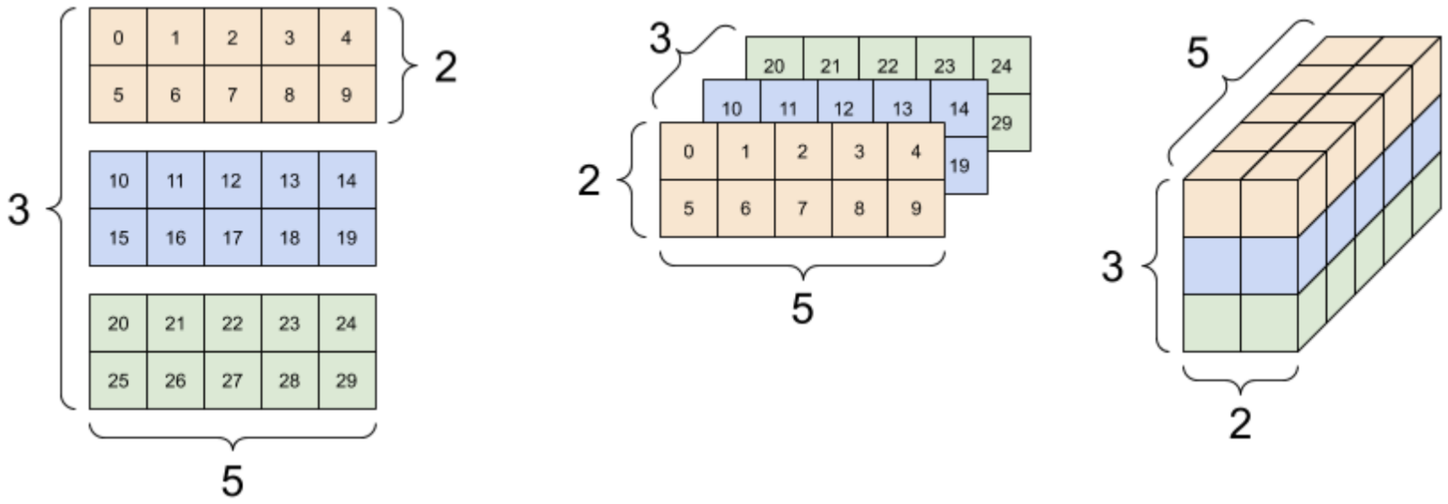
```
print(t.shape)  # para obter o shape do tensor
```

PyTorch

Tensores

Shape

- Há muitas maneiras de **visualizar** um tensor com **mais de dois** eixos.



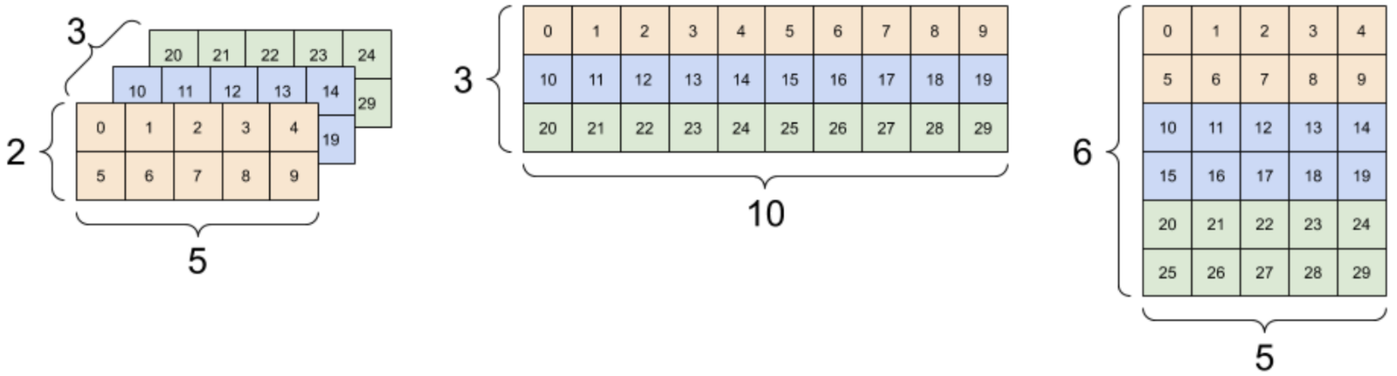
- No PyTorch, a forma de um tensor é **sempre conhecida**: t.shape (ou t.size()) devolve um torch.Size.

Tensores

Reshape

- É possível **alterar** a shape de um torch.Tensor através do torch.reshape (ou t.reshape / t.view).
- O **número de elementos** de um tensor é o **produto** dos tamanhos de todas as suas shapes.
- Normalmente, o torch.reshape é usado para **combinar** ou **dividir** eixos adjacentes (ou adicionar/remover).

```
rank_three_tensor = torch.ones([3, 2, 5])            #3x2x5 (30 elementos)
matrixA = torch.reshape(rank_three_tensor, [6, 5])   #Reshape para uma matriz 6x5 (30 …)
matrixB = torch.reshape(matrixA, [3, -1])            #Reshape para uma matriz 3x10 (30 …)
                                                     #-1: calcula o tamanho da dimensão
```



Tensores

Reshape

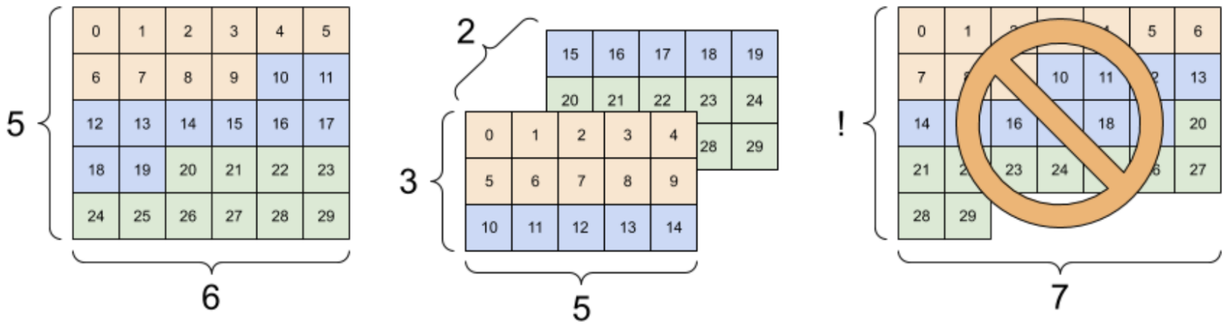
- Note que o **número de elementos** de reshaped Tensors tem de **corresponder** ao **número original** de elementos
- A remodelação "funcionará" para qualquer nova forma com o mesmo número total de elementos, mas não fará nada útil se não respeitar a ordem dos eixos.

```
# Maus exemplos: não façam isto!!

# Não é possível reordenar eixos com o reshape
print(torch.reshape(rank_3_tensor, [2, 3, 5]), "\n")

# Isto é uma confusão
print(torch.reshape(rank_3_tensor, [5, 6]), "\n")

# Isto não funciona de todo
try:
  torch.reshape(rank_3_tensor, [7, -1])
except Exception as e:
  print(f"{type(e).__name__}: {e}")
```



PyTorch

Tensores

Dtype

- Um torch.Tensor possui um **tipo de dados** especificado na propriedade **dtype.**
- **Cada elemento** no Tensor tem o **mesmo tipo de dados** e o tipo de dados é sempre conhecido.
- O PyTorch converte **números inteiros** do Python em **torch.int64** e **números de ponto flutuante** do Python em **torch.float32**.
- Ao contrário do NumPy, os torch.Tensors **não armazenam strings**: o texto tem de ser convertido em números primeiro.
- É possível **converter** torch.Tensors de um tipo de dados para outro utilizando t.to(dtype).

```
the_f64_tensor = torch.tensor([2.2, 3.3, 4.4], dtype=torch.float64)
# Converte para um uint8 e perde a precisão decimal
the_u8_tensor = the_f64_tensor.to(torch.uint8)
```

https://docs.pytorch.org/docs/stable/tensor_attributes.html

```
print(t.dtype)  # para obter o tipo de dados do tensor
```

PyTorch

Tensores

Device (CPU / GPU)

- Cada tensor vive num **device**: a CPU ou uma GPU ("cuda"; "mps" nos Mac).
- O **modelo e os dados** têm de estar no **mesmo device**; movem-se com **.to(device)**.
- Erro clássico: *Expected all tensors to be on the same device*.

EXEMPLO:

```
import torch

device = "cuda" if torch.cuda.is_available() else "cpu"
print(device)

t = torch.tensor([[1, 2], [3, 4]])
print(t.device)
t = t.to(device)   # devolve o tensor no device
print(t.device)
```

OUTPUT:

```
#sem GPU
cpu
cpu
cpu

#com GPU
cuda
cpu
cuda:0
```

Tensores

Criar tensores

- São criados com **torch.tensor()**, a partir de valores Python (números, listas) ou de arrays NumPy.
- Por omissão têm **requires_grad=False**: o autograd não calcula gradientes para eles.
- Usados para **dados e parâmetros fixos**, configurações e valores que não participam na aprendizagem.

SÍNTAXE:

```
torch.tensor(
 data,
 dtype=None,
 device=None,
 requires_grad=False
)
```

EXEMPLO:

```
import torch

a = torch.tensor([6, 7])
b = torch.tensor([[1,2], [3,4]])
x = torch.multiply(a, b)

print(x)
```

OUTPUT:

```
tensor([[ 6, 14],
        [18, 28]])
```

⚠ **Broadcasting** semelhante ao *numpy* (i.e., arrays de formas diferentes – o array mais pequeno é "transmitida" pelo array maior para que tenham formas compatíveis)

Tensores

Tensores como sequências

O PyTorch fornece um conjunto de funções para criar **tensores com padrões ou sequências**:

Função	Descrição	Exemplo
`torch.zeros(size, dtype)`	Tensor preenchido com zeros	`torch.zeros([2,3]) → [[0,0,0],[0,0,0]]`
`torch.ones(size, dtype)`	Tensor preenchido com uns	`torch.ones([2,3]) → [[1,1,1],[1,1,1]]`
`torch.zeros_like(tensor)`	Tensor de zeros com a forma e tipo de outro tensor	`torch.zeros_like(t) → [[0,0],[0,0]] # t é 2×2`
`torch.ones_like(tensor)`	Tensor de uns com a forma e tipo de outro tensor	`torch.ones_like(t) → [[1,1],[1,1]] # t é 2×2`
`torch.full(size, fill_value)`	Tensor preenchido com valor específico	`torch.full([2,2], 7) → [[7,7],[7,7]]`
`torch.linspace(start, end, steps)`	Sequência linear de steps valores entre start e end	`torch.linspace(0.0, 1.0, 5) → [0.0, 0.25, 0.5, 0.75, 1.0]`
`torch.arange(start, end, step)`	Sequência de valores inteiros ou float	`torch.arange(0,10,2) → [0,2,4,6,8]`

Tensores

Tensores como sequências

O PyTorch fornece um conjunto de funções para criar **tensores com padrões ou sequências**:

EXEMPLO:

```
import torch

zeros = torch.zeros([2,2])
ones = torch.ones([2,3])
filled = torch.full([2,2], 9)
seq = torch.arange(0,5)

print("Zeros:\n", zeros)
print("Ones:\n", ones)
print("Filled:\n", filled)
print("Sequence:\n", seq)
```

```
Zeros:
 tensor([[0., 0.],
         [0., 0.]])
Ones:
 tensor([[1., 1., 1.],
         [1., 1., 1.]])
Filled:
 tensor([[9, 9],
         [9, 9]])
Sequence:
 tensor([0, 1, 2, 3, 4])
```

Tensores

Tensores aleatórios

O PyTorch também permite criar **tensores** com **valores aleatórios**:

Função	Descrição	Exemplo
`torch.rand(size)`	Uniforme entre 0 e 1 (multiplicar para outro intervalo)	`torch.rand([2,2]) * 10`
`torch.normal(mean, std, size)`	Normal (Gaussiana)	`torch.normal(0.0, 1.0, [2,2])`
`torch.nn.init.trunc_normal_(tensor, mean, std)`	Normal truncada (preenche um tensor existente)	`torch.nn.init.trunc_normal_(torch.empty(2,2))`
`torch.randperm(n)`	Permutação aleatória de índices (para baralhar um tensor)	`x[torch.randperm(4)]`

Tensores

Tensores aleatórios

O PyTorch também permite criar **tensores** com **valores aleatórios**:

EXEMPLO:

```
import torch

torch.manual_seed(42)
u = torch.rand([2,2]) * 10
n = torch.normal(0.0, 1.0, [2,2])
t = torch.nn.init.trunc_normal_(torch.empty(2,2))

print("Uniforme:\n", u)
print("Normal:\n", n)
print("Truncada:\n", t)
```

OUTPUT:

```
Uniforme:
 tensor([[8.8227, 9.1500],
        [3.8286, 9.5931]])
Normal:
 tensor([[ 0.2345,  0.2303],
        [-1.1229, -0.1863]])
Truncada:
 tensor([[ 0.5349, -0.6380],
        [ 0.4617,  0.2674]])
```

Tensores

Operações

#Ops

torch.unique – encontra elementos únicos num tensor

torch.eq - devolve o valor de verdade de (x == y) elemento a elemento

torch.gt - devolve o valor de verdade de (x > y) elemento a elemento

torch.sort – ordena um tensor

torch.masked_select – aplica uma máscara booleana ao tensor

torch.add - devolve x + y elemento a elemento.

...

```
a + b      # torch.add
a - b      # torch.sub
a * b      # torch.mul
a / b      # torch.div
```

https://docs.pytorch.org/docs/stable/torch.html

Tensores

Operações Matemáticas

Bastante similares ao numpy

#Arithmetic Ops
torch.abs
torch.neg
torch.sign
torch.reciprocal
torch.square
torch.eq
torch.gt
torch.sqrt
torch.pow
torch.exp

https://docs.pytorch.org/docs/stable/torch.html#math-operations

Tensores

Integração com o NumPy

- Os tensores (em CPU) convertem-se em arrays NumPy com t.numpy(), e os arrays em tensores com torch.from_numpy(), partilhando a mesma memória
- As operações matemáticas do PyTorch convertem escalares Python em tensores (broadcasting); os arrays NumPy devem ser convertidos primeiro

```
#a tensor
a = torch.tensor([[1, 2], [3, 4]])
print(a)

#broadcasting support
b = torch.add(a, 1)
print(b)

#operator overloading is supported
print(a * b)
```

```
tensor([[1, 2],
        [3, 4]])

tensor([[2, 3],
        [4, 5]])

tensor([[ 2,  6],
        [12, 20]])
```

Tensores

Integração com o NumPy

- torch.Tensor.numpy() devolve o valor do objeto como um ndarray NumPy
- Os Tensores são iteráveis (utilize torch.equal ou torch.eq para comparar os Tensores)

```
#using numpy
import numpy as np

#a and b from previous slide
c = np.multiply(a.numpy(), b.numpy())
print(c)

#obtain numpy value from a tensor
print(a.numpy())

#iterable
for i in torch.tensor([1, 2, 3]) :
    print(i)
```

```
[[ 2  6]
 [12 20]]


[[1 2]
 [3 4]]



tensor(1)
tensor(2)
tensor(3)
```

Tensores

Vetorização

- **Vetorizar** = usar **operações sobre tensores inteiros** em vez de ciclos for.
- O cálculo é feito em **paralelo** (na CPU ou na GPU), e não elemento a elemento.
- É o que as camadas de uma rede neuronal fazem: **y = Wx + b**.

EXEMPLO:

```
import torch, time

a = torch.rand(1_000_000)
b = torch.rand(1_000_000)

start = time.time()
c = torch.dot(a, b)                # vetorizado
print('Vetorizado: %.1f ms' %(1000*(time.time()-start)))

d = 0
start = time.time()
for i in range(1_000_000):         # com ciclo for
    d += a[i] * b[i]
print('Com ciclo: %.1f ms' %(1000*(time.time()-start)))
```

OUTPUT:

```
Vetorizado: 2.5 ms
Com ciclo: 8947.6 ms

#mais de 3000 vezes mais lento!
```

PyTorch

Tensores

Criar tensores

Qual a limitação destes tensores?

⚠ Por omissão, os tensores **não registam** as operações em que participam (requires_grad=False): o autograd não calcula gradientes para eles e, por isso, **não podem ser otimizados** durante o treino!

💡 **Dica:**

- Utilize **torch.tensor()** para **valores fixos e não treináveis** como:
  - Parâmetros matemáticos fixos
  - Escalas, limites, coeficientes constantes
  - Configurações do modelo

- Utilize **requires_grad=True** (ou **nn.Parameter**) para pesos e parâmetros que devem ser otimizados ou que mudam durante o treino.

Aplicação Prática

Ficha de exercícios 3 – Parte I



Blackboard

Autograd

Tensores treináveis

- Representam valores **mutáveis**, geralmente **pesos** de modelos ou **parâmetros** que **mudam durante o treino**.
- São tensores com **requires_grad=True** (ou nn.Parameter): o autograd calcula os seus gradientes.

SÍNTAXE:

```
torch.tensor(
 data,
 dtype=None,
 device=None,
 requires_grad=True
)

torch.nn.Parameter(
 data=None,
 requires_grad=True
)
```

EXEMPLO:

```
import torch

w = torch.tensor(5.0, requires_grad=True)
print("w:", w)
```

OUTPUT:

```
w: tensor(5., requires_grad=True)
```

⚠ Mas porquê torch.**t**ensor e nn.**P**arameter? (t minúsculo e P maiúsculo)
Porque **torch.tensor** é uma **função** e **nn.Parameter** é uma **classe** (subclasse de Tensor)!

Grafos

O **PyTorch** utiliza um **grafo computacional dinâmico** para registar as operações aplicadas aos tensores e as dependências entre elas.

Porquê?

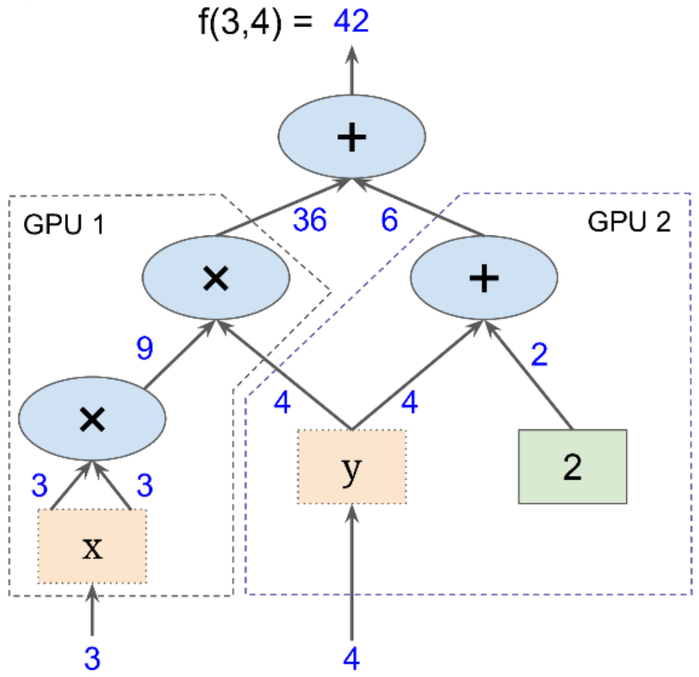
- Diferenciação automática - percorrer o grafo de trás para a frente para calcular os gradientes
- Dividir a computação - em partes pequenas e diferenciáveis
- Flexibilidade - o grafo é reconstruído em cada execução, pelo que pode depender de condições (if) e ciclos (for) normais do Python
- Facilitar a depuração - o código corre linha a linha, podendo usar-se print() ou um debugger
- Muitos modelos comuns de *machine learning* são ensinados e visualizados como grafos (Redes Neurais!!!)

Isto leva a um **modelo de programação imperativo,** no qual se escreve código Python normal: o grafo é criado durante o **forward pass** e percorrido no **backward pass** para **calcular os gradientes**.

Grafos

Num grafo:

- Os **nodos** representam as **unidades de computação**;
- As **arestas** representam os **dados consumidos** ou **produzidos** por uma computação.
- Os **dados** (**tensores**!) **fluem** pelo grafo.



PyTorch

Grafos

O que é o grafo computacional do PyTorch?

- É uma forma de representar cálculos matemáticos no PyTorch (um grafo acíclico dirigido) que contém um conjunto de objetos **Function** (acessíveis em *grad_fn*) (unidades de computação - **nós**) e objetos **torch.Tensor** (unidades de dados - **arestas**).

  Operações (nós): São os cálculos ou funções do grafo, como a adição, a multiplicação, etc.

  Tensores (arestas): São as matrizes multidimensionais de dados que fluem entre operações.

- O grafo só é registado para tensores com **requires_grad=True** e é **recriado de raiz** em cada execução (*forward pass*).

- O PyTorch utiliza **grafos computacionais dinâmicos** (*define-by-run*), o que significa que o grafo é construído à medida que o código é executado.

Autograd

O que é o torch.autograd?

- É o motor de **diferenciação automática** do PyTorch: **regista** as operações efetuadas sobre os tensores com **requires_grad=True**.
- Ao chamar **loss.backward()**, percorre o grafo no sentido inverso e **acumula os gradientes** no atributo **.grad** de cada tensor.
- Não existem sessões: o código é executado **de imediato**, linha a linha (**execução eager**).
- Com **torch.no_grad()** desativa-se o registo de operações (útil na **avaliação/inferência**).

Autograd

Exemplo

EXEMPLO:

```
import torch

x = torch.tensor(3.0)                          # dado
w = torch.tensor(2.0, requires_grad=True)  # peso
b = torch.tensor(1.0, requires_grad=True)  # bias

#forward: o grafo é construído aqui
y = w * x + b
loss = (y - 10) ** 2
print(loss)

#backward: calcula os gradientes
loss.backward()
print(w.grad)
print(b.grad)
```

OUTPUT:

```
#print(loss) guarda a operação que o criou
tensor(9., grad_fn=<PowBackward0>)

#print(w.grad) = 2 * (y - 10) * x
tensor(-18.)

#print(b.grad) = 2 * (y - 10)
tensor(-6.)
```

Autograd

Descida do gradiente à mão

EXEMPLO:

```
#w, b e os gradientes do slide anterior
lr = 0.01

#atualiza os pesos fora do grafo
with torch.no_grad():
    w -= lr * w.grad
    b -= lr * b.grad
print(w)
print(b)

#os gradientes acumulam: é preciso limpá-los!
w.grad.zero_()
b.grad.zero_()
```

OUTPUT:

```
#print(w) = 2.0 - 0.01 * (-18)
tensor(2.1800, requires_grad=True)

#print(b) = 1.0 - 0.01 * (-6)
tensor(1.0600, requires_grad=True)
```

É exatamente isto que **optimizer.step()** e **optimizer.zero_grad()** fazem por nós no ciclo de treino.

Controlar aleatoriedade

Sem seeds



⚠ **Controlar a aleatoriedade é crucial para obter resultados estáveis e replicáveis!**

```
#Without seeds
print(torch.rand(1).numpy())
print(torch.rand(1).numpy())
```

```
[0.9330233]
[0.59430563]

#closing the program and then running it again
[0.06516641]
[0.73865205]
```

⚠ **Obtemos resultados diferentes para cada chamada e a cada repetição do programa.**

Controlar aleatoriedade

Seed global



⚠ **Controlar a aleatoriedade é crucial para obter resultados estáveis e replicáveis!**

```
#Global seed

torch.manual_seed(1234)

print(torch.rand(1).numpy())
print(torch.rand(1).numpy())
```

```
[0.02897924]
[0.40189856]

#closing the program and then running it again
[0.02897924]
[0.40189856]
```

⚠ Obtemos **resultados diferentes** para cada **chamada** mas a **mesma sequência** para cada **repetição do programa**.

A razão pela qual obtemos **0.40 em vez de 0.03** na segunda chamada de **torch.rand** é que o gerador global **avança o seu estado interno** a cada número gerado.

Controlar aleatoriedade

➡ Reprodutibilidade na prática



⚠ **Controlar a aleatoriedade é crucial para obter resultados estáveis e replicáveis!**

Para obter resultados reprodutíveis é preciso fixar **todas as fontes** de aleatoriedade:

- **As sementes globais** - do Python, do NumPy e do PyTorch (**torch.manual_seed(x)**)
- **Um gerador próprio (*torch.Generator*)** - para quem baralha os dados (DataLoader, random_split)
- **Na GPU** - algumas operações não são determinísticas; torch.use_deterministic_algorithms(True) força versões determinísticas

```
#Global seeds
random.seed(42)
np.random.seed(42)
torch.manual_seed(42)
```

```
#Generator (seed local)
g = torch.Generator().manual_seed(42)
loader = DataLoader(ds, shuffle=True,
                    generator=g)
```

PyTorch

Aplicação Prática

Ficha de exercícios 3 – Parte II

